



A Retrieval-Augmented Generation (RAG) Based DevOps Assistant using LangChain and Ollama

Project Documentation

Developed By

Vihaan Agarwal

Technologies Used

- Python
 - Streamlit
 - LangChain
 - FAISS
 - Ollama
 - Phi3 Mini
 - HuggingFace Embeddings
-

1. Problem Statement

Large Language Models (LLMs) are capable of generating human-like responses but often suffer from limitations such as hallucination, lack of domain-specific knowledge, and inability to access private organizational documents.

Traditional AI chatbots rely entirely on pretrained knowledge and therefore cannot reliably answer questions from custom enterprise documentation.

The objective of this project is to develop a Retrieval-Augmented Generation (RAG) based chatbot capable of answering user queries from cloud infrastructure and DevOps related documents with grounded and context-aware responses.

The system combines semantic retrieval with modern language models to improve factual accuracy and reduce hallucinations.

The chatbot is designed to:

- Process cloud and DevOps related documents
- Convert document content into vector embeddings

- Retrieve semantically relevant information
 - Generate grounded responses using retrieved context
 - Provide a modern GPT-style interface using Streamlit
 - Prevent hallucinated responses using strict prompt engineering
-

2. Objectives

The primary objectives of the project are:

- Build a fully functional RAG pipeline using LangChain
- Enable question answering over custom DevOps documents
- Integrate vector database storage using FAISS
- Reduce hallucinations through context-constrained prompting
- Implement a modern conversational chatbot interface
- Create a scalable and modular AI architecture
- Implement session-based chat history
- Develop an enterprise-style UI experience

Secondary objectives include:

- Understanding Retrieval-Augmented Generation systems
 - Learning embedding-based semantic search
 - Understanding prompt engineering techniques
 - Exploring local LLM deployment using Ollama
 - Building production-oriented AI workflows
-

3. Technology Stack

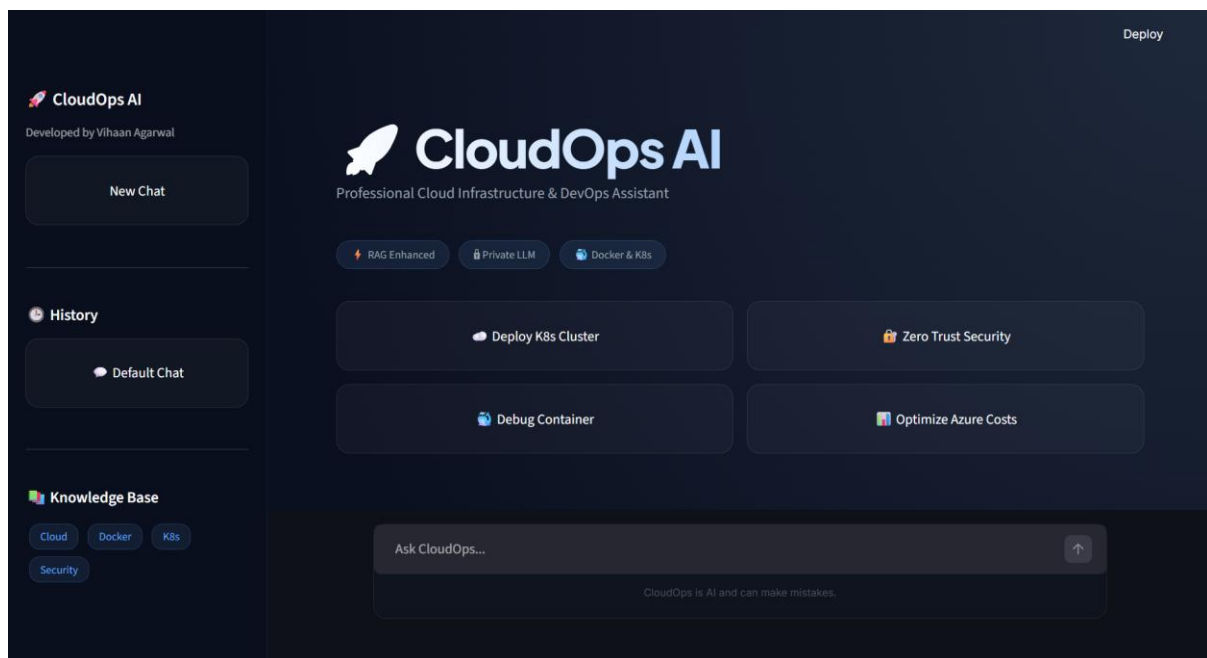
Component	Technology
Programming Language	Python
Frontend Interface	Streamlit
LLM Framework	LangChain
Language Model Runtime	Ollama
LLM Model	Phi3 Mini
Vector Database	FAISS
Embedding Model	HuggingFace all-MiniLM-L6-v2
Semantic Retrieval	Similarity Search
Session Management	Streamlit Session State
Deployment Environment	Local System

4. System Architecture

The architecture follows a Retrieval-Augmented Generation workflow.

Workflow

1. User enters a query through the Streamlit interface
2. Query is embedded using HuggingFace embeddings
3. FAISS performs semantic similarity search
4. Relevant chunks are retrieved from the vector database
5. Retrieved context is injected into the prompt template
6. Prompt is sent to Phi3 Mini through Ollama
7. LLM generates a grounded response
8. Response is displayed in the chatbot interface
9. Chat history is stored using session state



5. Architectural Components

Streamlit User Interface

Provides the interactive GPT-style frontend experience.

Features include:

- Sidebar chat history
- Floating centered input bar
- Modern message bubbles
- Retrieved context display
- Session-based interaction

Embedding Layer

Converts text chunks into dense vector representations using HuggingFace embeddings.

FAISS Vector Database

Stores vector embeddings and enables semantic similarity retrieval.

Retriever

Retrieves top-k semantically relevant chunks from the vector database.

Prompt Engineering Layer

Constrains the LLM to answer strictly from retrieved context.

Ollama + Phi3 Mini

Generates grounded responses using the local language model.

6. Development Phases

Phase 1 — Local Chatbot Development

- Built initial chatbot using Streamlit
- Integrated Ollama with Phi3 Mini
- Implemented basic conversational interface

Phase 2 — LangChain Integration

- Added LangChain orchestration
- Structured reusable pipeline components
- Integrated prompt templates

Phase 3 — Vector Database Integration

- Implemented FAISS vector storage

- Generated semantic embeddings
- Enabled similarity search retrieval

Phase 4 — Modern UI Enhancement

- Added GPT-style user interface
- Implemented sidebar history
- Added floating centered chat input
- Improved visual consistency and alignment

Phase 5 — Hallucination Prevention

- Added strict prompt engineering
- Implemented retrieval threshold filtering
- Added fallback response handling
- Reduced irrelevant AI-generated answers

Phase 6 — Evaluation and Testing

- Tested cloud and DevOps related queries
- Evaluated retrieval precision
- Verified hallucination prevention behavior
- Optimized retrieval thresholds

7. Hallucination Prevention Strategy

One of the major goals of the project was reducing hallucinated responses.

The following mitigation techniques were implemented:

- Strict context-only prompt constraints
- Similarity threshold filtering
- Fallback response system
- Context validation checks

Fallback response used:

"Sorry, I couldn't find relevant information about that in the uploaded documents."

This significantly reduced irrelevant and fabricated responses.

8. User Interface Features

The project includes a modern enterprise-style UI inspired by modern AI assistants.

Features

- Minimal professional design
 - Dark theme interface
 - Sidebar chat history
 - Floating chat input bar
 - Modern response cards
 - Retrieved context viewer
 - Responsive layout
 - Session-based conversation flow
-

9. Evaluation Results

Successful Retrieval Categories

Semantic Retrieval

The chatbot successfully answered:

- Kubernetes deployment questions
- Docker troubleshooting queries
- Cloud infrastructure concepts
- Networking related topics

Context Grounding

The chatbot answered questions using retrieved document content instead of pretrained knowledge.

Hallucination Prevention

The chatbot correctly rejected out-of-context questions such as:

- General knowledge unrelated to uploaded documents
- Random factual queries
- Non-cloud related questions

Current Performance Assessment

Component	Assessment
Streamlit UI	Stable
Retrieval Pipeline	Functional

Component	Assessment
Semantic Search	Good
Hallucination Prevention	Strong
Local LLM Integration	Stable
Session Memory	Functional
Production Readiness	Moderate

10. Challenges Faced

Retrieval Precision

Certain semantically complex queries occasionally retrieved partially relevant chunks.

Root Causes

- Embedding limitations
- Semantic mismatch
- Chunk fragmentation

Mitigation

- Adjusted retrieval threshold
- Optimized chunk retrieval logic
- Improved prompt constraints

Hallucinated Responses

The model initially answered using pretrained knowledge instead of retrieved context.

Solution

- Strict prompt engineering
 - Similarity filtering
 - Explicit fallback response handling
-

11. Current Limitations

The current implementation has the following limitations:

- No reranking pipeline
- No source citation system
- Limited conversational memory

- Single-domain optimization
 - No hybrid retrieval
 - No live PDF upload system
 - Limited multi-document support
-

12. Future Improvements

Planned improvements include:

Conversational Memory

Implement memory-aware retrieval for multi-turn conversations.

Source Citations

Display retrieved chunk references and page sources.

PDF Upload System

Allow dynamic PDF uploads directly from the UI.

Hybrid Search

Combine:

- Vector search
- Keyword search
- Metadata filtering

Streaming Responses

Implement token streaming for real-time response generation.

Multi-Document Support

Allow simultaneous querying across multiple uploaded documents.

Deployment Optimization

- Docker containerization
- Cloud deployment
- API optimization
- Secure configuration management

13. Conclusion

This project successfully demonstrates the implementation of a Retrieval-Augmented Generation (RAG) based AI assistant using Streamlit, LangChain, FAISS, and Ollama.

The chatbot is capable of answering grounded questions related to cloud infrastructure and DevOps while significantly reducing hallucinated responses.

The project establishes a strong foundation for future enhancements involving:

- Advanced retrieval systems
- Conversational memory
- Source citations
- Multi-document support
- Production-grade deployment

The system demonstrates practical understanding of:

- RAG architecture
- Vector databases
- Semantic retrieval
- Prompt engineering
- Local LLM deployment
- AI system design

14. References

- [LangChain Documentation](#)
 - [Streamlit Documentation](#)
 - [Ollama Documentation](#)
 - [FAISS Documentation](#)
 - [HuggingFace Embeddings Documentation](#)
 - [Phi3 Mini Model Documentation](#)
-